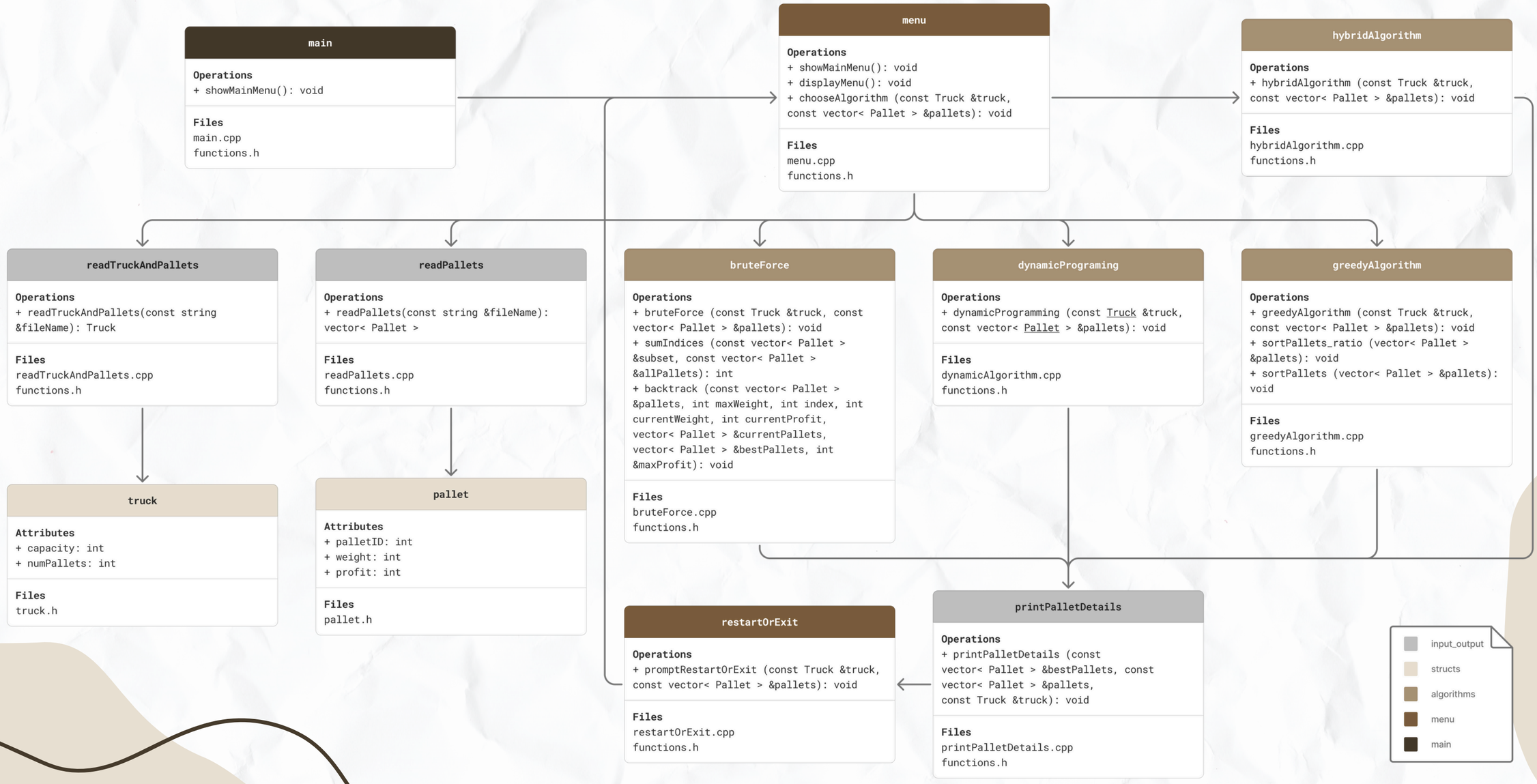
*Design of Algorithms (DA)*

# Delivery Truck Pallet Packing Optimization

T01_G05


Ana Catarina Monteiro de Sousa up202306419edu.fe.up.pt
Matilde de Sá Nogueira de Sousa up202305502edu.fe.up.pt

# Class Diagram



# Data Reading and Structuring

In this project, data on **pallets** and **trucks** is read from CSV files using C++ file I/O operations. The data is then processed and stored in suitable structures for efficient handling during the optimization process.

Each **truck** structure stores:

- Its maximum capacity (weight);
- The number of available pallets.

**Pallets** are stored in a vector, with each pallet containing:

- A unique ID;
- Its weight;
- The associated profit.

These structures support the implementation of algorithms that aim to maximize total profit while respecting the trucks' capacity limits.

# Description of Algorithms

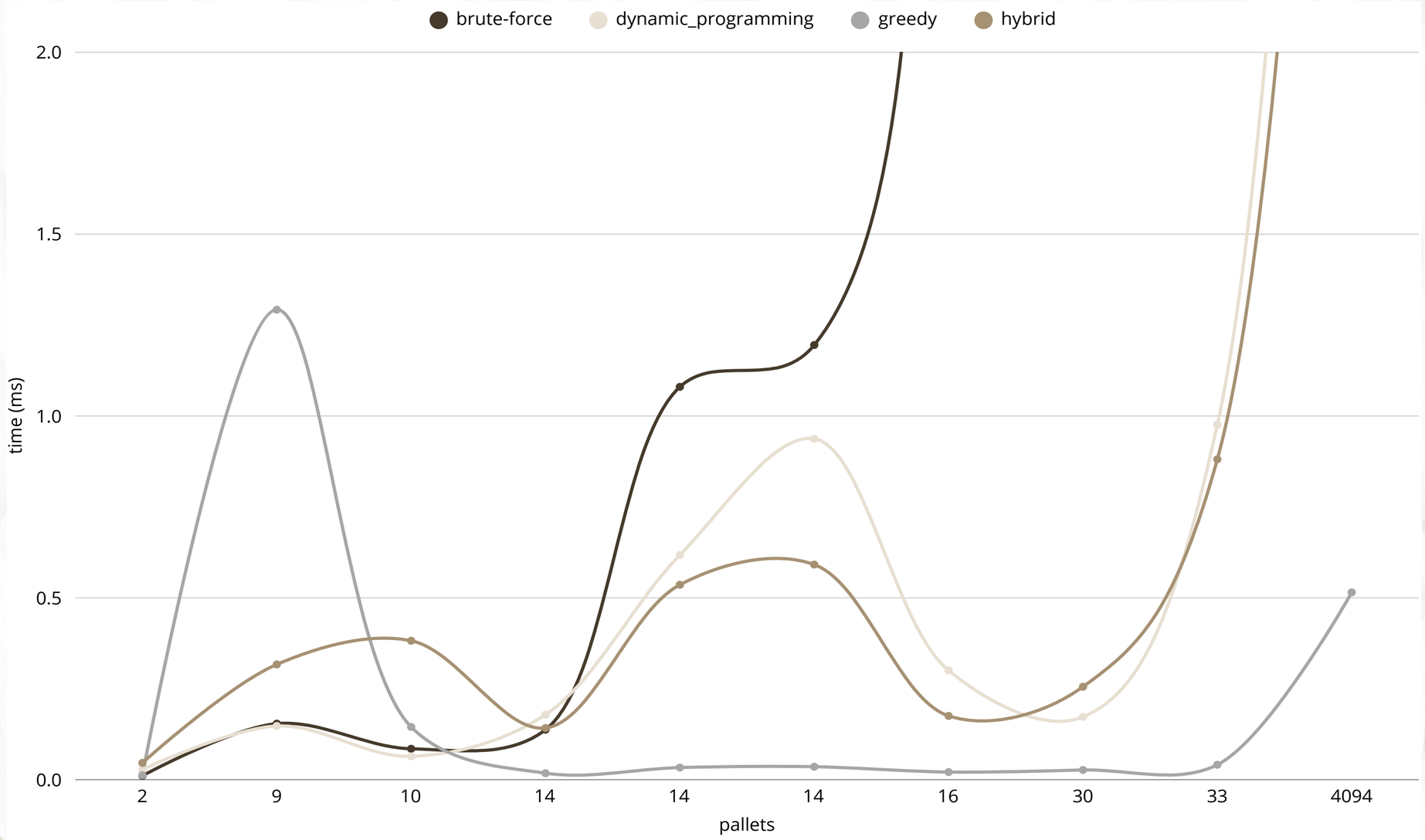
| Strategy | Description | Data Structures Used |
|---|---|---|
| Brute-force | Generates all subsets of pallets, checks weight constraints, and selects the subset with the highest profit. Exact but exponential time. | vector<Pallet> for subsets and pallet storage |
| Dynamic Programming | Classic 0/1 knapsack DP approach that builds a table of max profit by capacity. Optimal and efficient for moderate capacities. | 2D vector<vector<int>> DP table; vector<Pallet> for input |
| Greedy | Selects pallets based on the highest profit-to-weight ratio, until capacity is reached. Fast but not guaranteed optimal. | vector<Pallet> sorted by profit/weight ratio |
| Hybrid (DP + Greedy) | Combines DP on a reduced capacity subset and greedy to complete the solution, balancing accuracy and speed. | Smaller DP table (vector<int> or vector<vector<int>>); vector<Pallet> sorted for greedy |

# Description of Algorithms

| Strategy | Solution Type | Time Complexity | Space Complexity | Advantages | Disadvantages |
|---|---|---|---|---|---|
| Brute-force | Optimal | $O(2^n)$ | O(n) | Guarantees the best possible solution | Not feasible for large datasets |
| Dynamic Programming | Optimal | O(n × W) | O(n × W) | Efficient, suitable for moderate W | High memory usage for large capacities |
| Greedy | Approximate | O(n log n) | O(n) | Very fast and easy to implement | May fail to find the optimal solution |
| Hybrid (DP + Greedy) | Approximate/Partial | Worst-case: O(n × W)<br>Best-case: O(n log n) | O(n × W) | Balance between accuracy and speed | Results depend on choice of W |

n - number of pallets
W - truck capacity

# Execution Time per Pallet



# User Interface

The system features a terminal-based interactive menu, providing a simple and efficient user experience. The user is prompted to enter the dataset number, after which two CSV files are automatically loaded—one containing the truck data and another with pallet information. If the files are invalid or unavailable, the system prompts for a new selection.

Once the data is successfully loaded, the user can choose from four optimization algorithms: Brute-force, Dynamic programming, Greedy algorithm, Hybrid algorithm, as well as the option to exit the program.

Each algorithm aims to maximize profit by selecting the optimal combination of pallets, considering the truck's weight and volume constraints. The results are displayed directly in the terminal and include the selected pallets, total profit, truck capacity usage, and the time taken to execute the algorithm.

After presenting the results, the system prompts the user to choose whether to test another algorithm on the same dataset, select a new dataset, or exit the program. This allows for an easy and flexible comparison between different optimization approaches.

# Exemple of Use

The user starts the program and selects dataset number 10. They then choose the dynamic programming algorithm to optimize pallet loading. The system runs the algorithm and displays the results in the terminal, including maximum profit, number of pallets selected, pallet details, and execution time. Finally, the user opts to exit the program, which closes with a thank-you message.

```
Delivery Truck Pallet Packing Optimization

Please insert the number of the dataset to use: 10

SELECT ALGORITHM TO RUN

[1] Brute-force (exhaustive search)
[2] Dynamic programming
[3] Greedy algorithm
[4] Hybrid algorithm (Greedy + DP)
[5] Exit
--------------------------------------------------
Your choice: 2

DYNAMIC PROGRAMING

Maximum profit: 15
Number of pallets: 1
Execution time: 0.028125 ms
-------------------- PALLETS: --------------------
```

| Pallet | Weight | Profit |
|---|---|---|
| 2 | 16 | 15 |

```
NEXT ACTION

[1] Test another dataset
[2] Run another algorithm on the same dataset
[3] Exit
--------------------------------------------------
Your choice: 3

Thank you for using the program!
```

# Highlights

## User-Friendly Interface

The terminal-based interactive menu was designed to be intuitive and straightforward, allowing users with no technical background to easily load datasets, select algorithms, and view results.

## Hybrid Algorithm

A novel approach combining greedy heuristics with dynamic programming, providing a balance between solution quality and computational efficiency. This hybrid method enhances performance on larger datasets where pure exhaustive or dynamic programming methods may be impractical.

# Last Comments

## *Main Difficulties*

Implementing and optimizing the hybrid algorithm, balancing efficiency and solution quality.

Challenges in the implementation of backtracking, particularly in managing combinations and controlling complexity.